PlantGuard

Détection de maladies des plantes par Deep Learning et plateforme MLOps

Champ	Détail
Équipe	Raboudi Ibrahim · Othmen Chtioui · Yasmine Abdennadher
Établissement	Tek-Up University — Data Science Engineering
Livrable	Cadrage du projet de Deep Learning
Envoi	haythem.ghazouani@tekup.de
Date	13 octobre 2026

01 · Sujet et problématique

Sujet. PlantGuard est un système qui reconnaît la maladie d'une plante à partir d'une simple photo de feuille. Le projet couvre toute la chaîne : préparation des données, entraînement de réseaux de neurones convolutifs (CNN) et d'un modèle par transfert d'apprentissage (ResNet50), suivi des expériences, API d'inférence, tableau de bord web et monitoring.

Intérêt. Selon la FAO, jusqu'à 40 % des cultures mondiales sont perdues chaque année à cause des organismes nuisibles et des maladies. Un diagnostic précoce, accessible depuis un smartphone, aide les agriculteurs qui n'ont pas toujours accès à un expert. Du point de vue technique, le sujet combine vision par ordinateur, classification multiclasse déséquilibrée et industrialisation MLOps.

Cas d'usage précis. Un utilisateur dépose ou capture une image de feuille ; l'application renvoie la classe prédite (espèce + maladie ou « saine »), un score de confiance et le temps de réponse.

Problématique. Comment identifier de façon fiable, rapide et reproductible la maladie d'une plante parmi 38 classes à partir d'une image de feuille, et comment industrialiser ce modèle (suivi, déploiement, monitoring) pour qu'il reste utilisable en conditions réelles ?

Axe technique. Classification d'images, apprentissage par transfert et fine-tuning, pipeline MLOps complet.

02 · Objectifs

Objectif général. Concevoir, entraîner, évaluer et déployer un système de diagnostic de maladies de plantes par images, traçable de bout en bout (données → modèle → API → interface → monitoring).

Objectifs spécifiques et mesurables :

#	Objectif spécifique	Indicateur de réussite
OS1	Valider et préparer le dataset : détection des images corrompues, découverte automatique des classes, split stratifié sans fuite, augmentation reproductible.	0 image illisible dans les splits ; répartition 70/15/15 stratifiée, graine fixée.
OS2	Entraîner un CNN de référence from scratch, puis ResNet50 en transfert et en fine-tuning, et comparer les résultats.	Macro-F1 ≥ 0,95 sur le jeu de test pour le meilleur modèle ; comparaison chiffrée des 3 approches.
OS3	Tracer chaque expérience avec MLflow et rendre le pipeline reproductible (DVC).	Chaque run enregistre paramètres, métriques et artefacts ; pipeline relançable en une commande.
OS4	Exposer le modèle via une API FastAPI (prédiction, santé, infos modèle).	Endpoints /predict, /health, /model-info ; latence p95 < 500 ms sur CPU ; validation des entrées.
OS5	Fournir un tableau de bord React et un monitoring (latence, erreurs, dérive), conteneurisé et testé en CI.	Upload d'image fonctionnel ; docker compose up ; workflow CI (lint + tests + build) au vert.

03 · Dataset

Champ	Détail
Nom	PlantVillage Dataset
Source	Hughes & Salathé (2015), projet PlantVillage (Penn State University). Article : « An open access repository of images on plant health to enable the development of mobile disease diagnostics », arXiv:1511.08060.
Licence	Page Kaggle du miroir : CC BY-NC-SA 4.0 (usage non commercial, attribution, partage à l'identique). Compatible avec un usage académique ; à citer dans le rapport final.
Taille	≈ 54 305 images (version color), environ 0,8 Go.
Format	Images JPEG RGB, 256×256 pixels, feuilles isolées sur fond uniforme (prises en laboratoire).
Labels	38 classes = 14 espèces (pomme, myrtille, cerise, maïs, vigne, orange, pêche, poivron, pomme de terre, framboise, soja, courge, fraise, tomate) ; 26 classes malades et 12 classes saines. Le label est le nom du dossier, au format Espece_Maladie.

04 · Exploration

Les statistiques ci-dessous sont les effectifs publiés pour la version color. Le script de validation du projet (Phase 1) les recalcule sur la copie locale à chaque exécution.

Statistique	Valeur
Images au total	54 305
Nombre de classes	38
Images par classe : moyenne / médiane	1 429 / 1 092
Classe la plus grande	Orange · Huanglongbing — 5 507 images
Classe la plus petite	Potato · Healthy — 152 images
Rapport max/min	≈ 36 : 1
Classes de tomate (10 classes)	18 160 images, soit 33 % du dataset
Classes « saines » (12 classes)	15 084 images, soit 28 % du dataset

05 · Qualité des données

Point de contrôle	Méthode	Constat et traitement
Valeurs manquantes	Dataset d'images : pas de colonnes. Contrôle des dossiers vides et des classes sans image.	Les labels viennent des noms de dossier, donc aucun label manquant ; classes découvertes automatiquement.
Corruption	Ouverture et vérification de chaque fichier avec Pillow (verify) ; contrôle de l'extension et du mode RGB.	Les fichiers illisibles sont écartés et listés dans un rapport avant tout split.
Doublons	Hash exact (MD5) puis hash perceptuel pour les quasi-doublons.	PlantVillage contient plusieurs vues et variantes d'une même feuille : un split aléatoire simple peut faire fuiter des quasi-doublons entre train et test. Déduplication avant le split.
Déséquilibre	Comptage par classe : rapport max/min ≈ 36 : 1 ; 5 classes sous 400 images.	Split stratifié, pondération des classes ou sampler pondéré, augmentation ciblée, et métrique principale = macro-F1 (pas seulement l'accuracy).
Biais d'acquisition	Inspection visuelle ; fond uniforme sur toutes les images.	Des travaux publiés montrent que le fond seul peut suffire à classer ; on utilise l'augmentation (recadrage, couleur) et on teste sur des photos hors dataset.

06 · Faisabilité

Tâche de Deep Learning. Classification d'images multiclasse (38 classes). Entrée redimensionnée à 224×224, normalisation par les statistiques ImageNet.

Approches envisagées :

Approche	Description
Baseline CNN	CNN from scratch (blocs convolution + BatchNorm + ReLU + pooling, puis couches denses) : point de comparaison.
Transfert	ResNet50 pré-entraîné sur ImageNet, tête de classification remplacée (38 sorties), couches du backbone gelées.
Fine-tuning	Dégel du dernier bloc (layer4) avec un taux d'apprentissage faible, pour gagner en précision.

Prétraitement envisagé. Validation des fichiers, déduplication, split stratifié 70/15/15 avec graine fixe, augmentation sur le train uniquement (flip, rotation, variation de luminosité et de couleur, recadrage aléatoire), validation et test sans augmentation.

Métriques :

- Qualité : macro-F1 (métrique principale), accuracy, top-3 accuracy, précision et rappel par classe, matrice de confusion.
- Exploitation : latence d'inférence (p50/p95), taux d'erreur de l'API, dérive des données par rapport aux statistiques de référence.

Ressources et GPU. ResNet50 compte environ 25,6 millions de paramètres. L'entraînement se fait sur un GPU de type T4 (Google Colab ou Kaggle), en précision mixte, avec un batch de 32. Un mode léger sur CPU (sous-ensemble de données, peu d'époques) permet de valider le pipeline en local ; l'inférence tourne sur CPU.

Limites et risques :

Risque	Impact	Parade
Images de laboratoire (fond uniforme)	Baisse de performance sur des photos prises au champ	Augmentation, test sur des images externes, annonce claire de la limite
Déséquilibre des classes	Classes rares mal reconnues	Macro-F1, pondération, augmentation
Quasi-doublons entre splits	Scores de test trop optimistes	Déduplication avant le split
Ressources GPU limitées	Entraînement lent	Précision mixte, Colab/Kaggle, mode léger CPU
Licence non commerciale	Pas d'usage commercial du dataset	Projet académique ; citation de la source
Périmètre : 14 espèces, pas de niveau de gravité	Le système ne couvre pas toutes les cultures	Documenter le périmètre dans le README

07 · Pipeline MLOps complet (généralisé)

Le pipeline suit le cycle de vie complet d'un modèle de Deep Learning industrialisé, de la donnée brute au monitoring en production, avec une boucle de ré-entraînement. Il est décrit de façon générale afin de rester valable quelle que soit la tâche (classification, détection, etc.).

Étape	Objectif	Outils / Emplacement
1. Collecte & versionning des données	Stocker les données de manière traçable et reproductible ; jamais de données lourdes dans Git.	DVC, dossier data/, archive/
2. Validation & prétraitement	Contrôle d'intégrité (fichiers corrompus), déduplication, split stratifié sans fuite, augmentation reproductible.	Scripts Python, Pillow, dossier ml/
3. Exploration & rapport de qualité	Statistiques, visualisations, détection du déséquilibre et des biais ; génération d'un rapport.	Scripts d'exploration, notebooks
4. Entraînement	Entraînement des modèles (baseline, transfert, fine-tuning) avec reproductibilité (graine fixe).	PyTorch, GPU (Colab/Kaggle), précision mixte
5. Suivi d'expériences	Journaliser paramètres, métriques et artefacts de chaque run pour comparer les expériences.	MLflow, dossier mlruns/
6. Évaluation & registre de modèles	Sélection du meilleur modèle sur la métrique principale ; export et métadonnées associées.	models/ (modèle .pt/ONNX + model_metadata.json)
7. Packaging & conteneurisation	Image Docker reproductible de l'API et de l'interface ; orchestration locale en une commande.	Docker, .dockerignore, docker-compose
8. Déploiement (serving)	API d'inférence avec validation des entrées, santé du service et informations du modèle.	Backend FastAPI, dossier backend/
9. Interface utilisateur	Application web permettant de téléverser une image et de consulter la prédiction.	Frontend React + Vite, dossier frontend/
10. Monitoring & alerting	Suivi de la latence, des erreurs et de la dérive des données ; déclenchement du ré-entraînement si seuil dépassé.	Dossier monitoring/
11. CI/CD	Intégration et déploiement continus : lint, tests, build et déploiement automatisés à chaque modification.	GitHub Actions, dossier .github/workflows
12. Boucle de ré-entraînement	Réinjecter les nouvelles données et les alertes du monitoring dans le pipeline (retour à l'étape 2).	Pipeline orchestré, DVC + MLflow

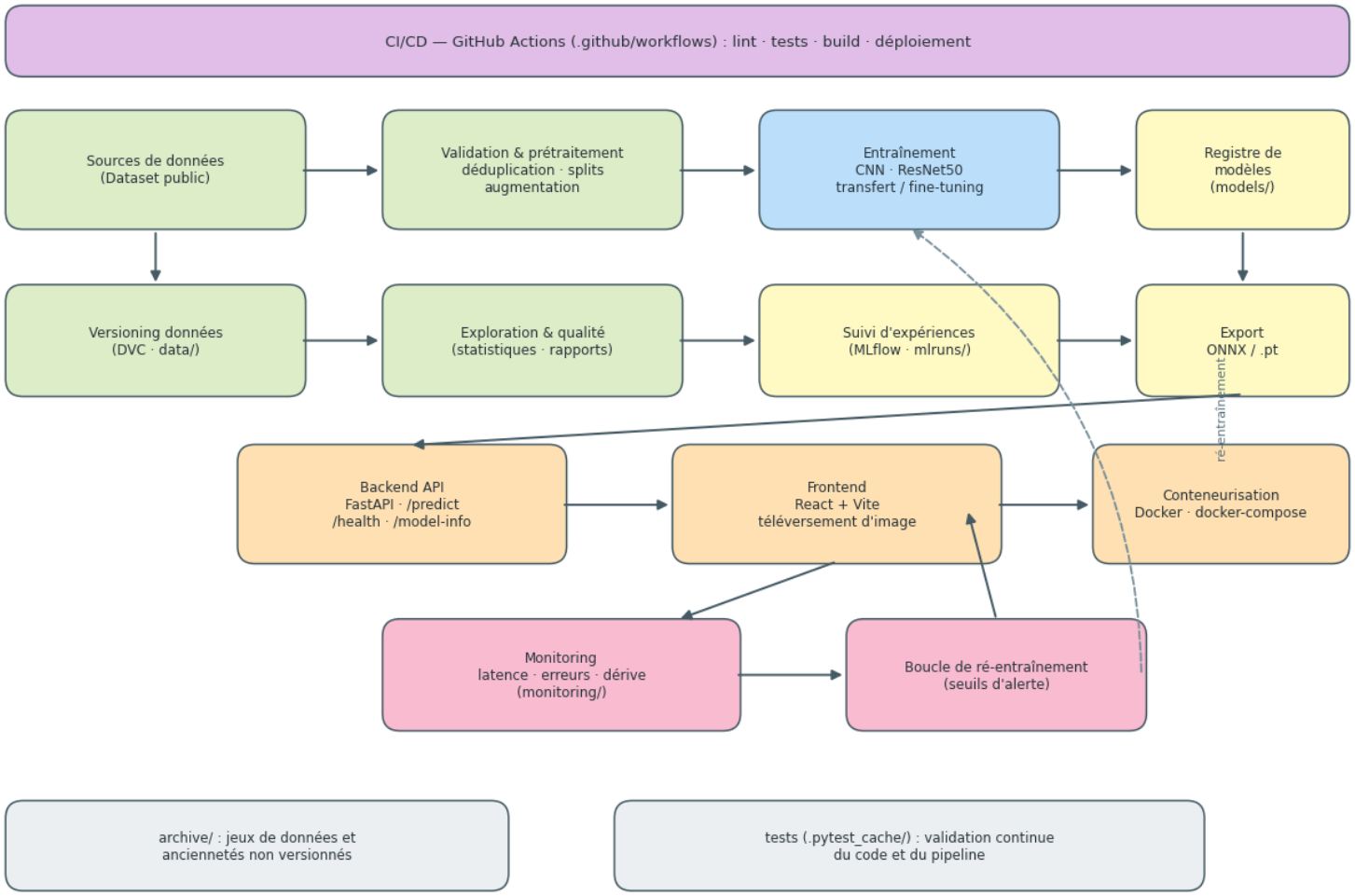


Schéma du pipeline et de l'architecture :

08 · Architecture du projet

L'architecture suit une organisation modulaire standard d'un projet MLOps, telle qu'elle apparaît dans l'explorateur du dépôt. Chaque dossier a un rôle précis dans le pipeline décrit en section 07.

Composant	Rôle dans l'architecture
.github/	Workflows GitHub Actions : lint, tests (pytest), build et déploiement continus à chaque push / pull request.
backend/	API de serving (FastAPI) : routes /predict, /health, /model-info ; validation des entrées ; chargement du modèle depuis models/.
frontend/	Interface web (React + Vite) : téléversement d'image, affichage de la classe prédite, du score de confiance et de la latence.
ml/	Cœur du pipeline Machine Learning : validation des données, prétraitement, augmentation, entraînement (baseline, transfert, fine-tuning) et scripts d'exploration.
data/	Données versionnées (DVC) : jeux bruts, jeux préparés et splits ; jamais versionnées directement dans Git.
models/	Registre local des modèles : artefacts (.pt, export ONNX) et métadonnées (model_metadata.json) du modèle en production.
mlruns/	Suivi d'expériences MLflow : journaux des runs (paramètres, métriques, artefacts) générés à l'entraînement.
monitoring/	Supervision en production : métriques de latence et d'erreurs de l'API, détection de dérive par rapport aux statistiques de référence, alertes.
archive/	Anciennes versions du dataset et artefacts obsolètes, conservés hors versioning (non suivis par Git).
Docker / .dockerignore / docker-compose	Conteneurisation de l'API, du frontend et du monitoring ; orchestration de l'ensemble en une commande (docker compose up).

Vue en couches :

- Couche données : sources publiques → data/ (DVC) → archive/ pour l'historique.
- Couche ML : ml/ (prétraitement, entraînement) → mlruns/ (suivi) → models/ (registre).
- Couche serving : backend/ (FastAPI) servant le modèle, frontend/ (React) pour l'utilisateur final.
- Couche exploitation : monitoring/ (observabilité) et .github/ (CI/CD) fermant la boucle de ré-entraînement.